**Name: Aziz Sayyad**
**Roll No: 381069**
**Batch: A3**

# Assignment No: 2

## Problem Statement:

Perform bag-of-words approach (count occurrence, normalized count occurrence), TF-IDF on data. Create embeddings using Word2Vec.

## Objective:

- To understand the bag-of-words (BoW) model and represent text as count vectors.
- To apply L2 normalization on count vectors to obtain normalized bag-of-words representations.
- To implement TF-IDF vectorization to capture term importance across documents.
- To create dense word embeddings using the Word2Vec model with Gensim.
- To compare sparse (BoW, TF-IDF) and dense (Word2Vec) text representation techniques.

## S/W Packages and H/W apparatus used:

- Operating System: Windows/Linux/MacOS
- Kernel: Python 3.x
- Tools: Google Colab / Jupyter Notebook / Anaconda
- Hardware: CPU with minimum 4GB RAM
- Libraries and packages used: NLTK, NumPy, Scikit-learn (CountVectorizer, TfidfVectorizer, normalize), Gensim (Word2Vec)

## Theory:

Text representation is a critical step in NLP that converts raw text into numerical formats suitable for machine learning algorithms. The main approaches range from simple frequency-based methods to sophisticated neural embedding models.

## Bag-of-Words (BoW):

The Bag-of-Words model represents a document as a vector of word counts based on a fixed vocabulary. Each document is described by the frequency of each vocabulary word appearing in it, ignoring grammar and word order.

- Count Occurrence: Each element in the vector is the raw integer count of how many times a word appears in the document.
- Normalized Count Occurrence: Raw count vectors are normalized using L2 normalization (dividing each vector by its Euclidean norm), so all document vectors have unit length. This removes bias caused by document length differences.

## TF-IDF (Term Frequency - Inverse Document Frequency):

TF-IDF is a statistical measure that evaluates how important a word is to a document within a corpus. It combines two metrics:

- TF (Term Frequency): How frequently a term appears in a document. Higher frequency means higher TF.
- IDF (Inverse Document Frequency): Penalizes terms that appear in many documents (common words like 'is', 'the'), giving higher weight to rare, informative terms.
- TF-IDF Score = TF x IDF. Words common across all documents get low scores; words unique to a document get high scores.

**Word2Vec:**

Word2Vec is a neural network-based model developed by Google that learns dense vector representations (embeddings) of words from large corpora. Unlike BoW and TF-IDF, Word2Vec captures semantic relationships between words -words with similar meanings have similar vectors. It offers two architectures:

- CBOW (Continuous Bag-of-Words): Predicts a target word from its surrounding context words.
- Skip-gram: Predicts surrounding context words given a target word. Better for rare words.

Key parameters include vector_size (dimensionality of embeddings), window (context window size), and min_count (minimum word frequency threshold).

**Methodology:**

- Install and import required libraries: NLTK, NumPy, Scikit-learn, and Gensim.
- Define a sample corpus of four documents covering NLP and AI topics.
- Apply CountVectorizer to build the vocabulary and generate count-based BoW matrix.
- Apply L2 normalization on the count matrix using sklearn's normalize function to produce normalized BoW vectors.
- Apply TfidfVectorizer to compute TF-IDF scores for all terms across documents.
- Tokenize documents using NLTK's word_tokenize (lowercase) to prepare input for Word2Vec.
- Train a Word2Vec model with vector_size=100, window=5, and min_count=1 on the tokenized corpus.
- Retrieve and display the 100-dimensional embedding vector for the word 'nlp'.
- Find and display the top 10 most similar words to 'ai' using cosine similarity on Word2Vec vectors.
- Analyze and compare the outputs of all four representation methods.

**Use Cases:**

- Use Case 1 - Document Classification: BoW count and TF-IDF vectors serve as features for machine learning classifiers (e.g., Naive Bayes, SVM) in spam detection and news categorization tasks.
- Use Case 2 - Semantic Search: Word2Vec embeddings enable semantic similarity search, allowing systems to retrieve documents related in meaning rather than just exact keyword matches.
- Use Case 3 - Recommendation Systems: TF-IDF representations help identify key topics in user reviews and product descriptions, improving content-based filtering in e-commerce and streaming platforms.

**Advantages:**

- BoW is simple to implement and interpretable; each dimension corresponds to a known vocabulary word.

- L2 normalization removes document length bias, making short and long documents comparable.
- TF-IDF automatically downweights common stop words and highlights domain-specific terms.
- Word2Vec captures semantic and syntactic word relationships that sparse methods cannot represent.
- Pre-trained Word2Vec models can be reused via transfer learning on small datasets.

**Limitations:**

- BoW ignores word order and context, losing important syntactic and semantic information.
- Both BoW and TF-IDF produce sparse, high-dimensional vectors that scale poorly with vocabulary size.
- TF-IDF does not capture semantic similarity; 'car' and 'automobile' are treated as unrelated.
- Word2Vec requires a sufficiently large corpus to learn meaningful embeddings; poor results on small datasets.
- Word2Vec produces a single static vector per word, unable to handle polysemy (context-dependent meanings).

**Applications:**

- Information retrieval and search engines: TF-IDF ranks document relevance to search queries.
- Sentiment analysis: BoW and TF-IDF features feed into classifiers for opinion mining from reviews.
- Machine translation: Word2Vec embeddings provide semantic word representations for neural MT models.
- Question answering systems: Word embeddings help match question semantics with relevant answer passages.
- Text clustering and topic modelling: TF-IDF vectors enable grouping of similar documents without supervision.

**Conclusion:**

This assignment demonstrated four key text representation techniques in NLP. The Bag-of-Words model provided a simple count-based representation, while L2 normalization addressed document length bias. TF-IDF improved upon raw counts by weighting terms according to their importance across the corpus. Word2Vec introduced dense semantic embeddings, capturing meaningful relationships between words such as similarity between 'ai' and contextually related terms. Each method has distinct strengths and trade-offs, and the choice depends on the downstream task -sparse methods suit traditional ML classifiers while dense embeddings are preferred for deep learning and semantic applications.